

human

Where are wood houses in Exeter ?

host_agent

User wants to find wood houses in Exeter. Provide the general steps to solve this query.

Tool Call: - Tool: send_message - Args: {"target": "planning_agent", "task_description": "User wants to find wood houses in Exeter. Provide the general steps to solve this query."}

planning_agent

The area is Exeter, so there will be 1 entry for Exeter. The entity to search for is wood houses, and we need to find as many entries as possible for wood houses in Exeter.

output steps: ["Find Exeter 1 area", "Search for wood houses in Exeter as many search results"]

Tool Call: - Tool: send_message - Args: {"target": "named_area", "task_description": "Find the area polygon for Exeter."}

named_area_agent

Find the area polygon for Exeter.

Tool Call: - Tool: call_os_ngd - Args: {"filters": null, "bbox": null, "polygon_or_point": true, "street_address": null, "filename": "exeter_area_polygon"}

Tool call result: Search results have been found. Artifact generated is : exeter_area_polygon, Description is : A geopandas dataframe containing named area data with no filters applied for the query (so remember no search is performed here so spurious entities present) Find the area polygon for Exeter..., count of records fetched : 98.

Tool Call: - Tool: send_message - Args: {"target": "data_analysis_agent", "task_description": "Filter the artifact 'exeter_area_polygon' to find the specific area polygon for Exeter. Save the filtered results as 'filtered_exeter_area_polygon'."}

data_analysis_agent

Filter the artifact 'exeter_area_polygon' to find the specific area polygon for Exeter. Save the filtered results as 'filtered_exeter_area_polygon'.

Tool Call: - Tool: generate_metadata_for_artifacts - Args: {"artifact_names": ["exeter_area_polygon"]}

Tool call result: Metadata generated for 'exeter_area_polygon'.

Tool Code Executor

```
import matplotlib
matplotlib.use('Agg')
def filter_exeter_area_polygon(data: list):
    gdf = data[0]
    # Filter for Exeter using multiple relevant columns for high search quality
    # Check for 'Exeter' in name1_text and relevant description/descriptiongroup
    filtered = gdf[((gdf['name1_text'].str.lower() == 'exeter') |
                    (gdf['name1_text'].str.lower().str.contains('exeter')) &
                    (gdf['descriptiongroup'].isin(['Settlement', 'Part Of Settlement', 'City', 'Town'])))]
    summary = f"Filtered to {len(filtered)} polygon(s) for Exeter area."
    artifact_name = 'filtered_exeter_area_polygon'
    artifact_description = 'GeoDataFrame containing only the area polygon(s) for Exeter, filtered by name and settlement-related description.'
    return [summary, artifact_name, artifact_description, filtered]
```

Filtered to 1 polygon for the Exeter area. The result is saved as 'filtered_exeter_area_polygon', which contains only the area polygon(s) for Exeter, filtered by name and settlement-related description.

Tool Call: - Tool: send_message - Args: {"target": "buildings", "task_description": "Search for wood houses within the area defined by 'filtered_exeter_area_polygon'."}

buildings_agent

Search for wood houses within the area defined by 'filtered_exeter_area_polygon'.

Tool Call: - Tool: call_os_ngd - Args: {"filters": null, "bbox": "filtered_exeter_area_polygon", "polygon_or_point": true, "street_address": null, "filename": "all_buildings_in_filtered_exeter_area.geojson"}

Tool call result: Multiple search results have been found in multiple datasets. Artifacts generated are : ['buildings_all_buildings_in_filtered_exeter_area.geojson', 'buildingline_all_buildings_in_filtered_exeter_area.geojson', 'buildingpart_all_buildings_in_filtered_exeter_area.geojson'], counts of records fetched are : [69169, 69342, 85480].

Tool Call: - Tool: send_message - Args: {"target": "data_analysis_agent", "task_description": "From 'buildings_all_buildings_in_filtered_exeter_area.geojson', filter for houses defined as buildings where buildinguse_addresscount_residential > 0 and buildinguse_addresscount_total = 1. Then, within these houses, identify those that are constructed primarily of wood or have wood as the main construction material. Save the filtered results as 'wood_houses_in_filtered_exeter_area.geojson'."}

data_analysis_agent

From 'buildings_all_buildings_in_filtered_exeter_area.geojson', filter for houses defined as buildings where buildinguse_addresscount_residential > 0 and buildinguse_addresscount_total = 1. Then, within these houses, identify those that are constructed primarily of wood or have wood as the main construction material. Save the filtered results as 'wood_houses_in_filtered_exeter_area.geojson'.

Tool Call: - Tool: generate_metadata_for_artifacts - Args: {"artifact_names": ["buildings_all_buildings_in_filtered_exeter_area.geojson"]}

Tool call result: Metadata generated for 'buildings_all_buildings_in_filtered_exeter_area.geojson'.

Tool Code Executor

```
import matplotlib
matplotlib.use('Agg')
def filter_wood_houses_in_filtered_exeter_area(data: list):
    import geopandas as gpd
    # Extract the input GeoDataFrame
    gdf = data[0]
```

```
# Step 1: Filter for houses (residential, single address)
houses = gdf[(gdf['buildinguse_addresscount_residential'] > 0) & (gdf['buildinguse_addresscount_total'] == 1)]
# Step 2: Identify those constructed primarily of wood
# Possible wood-related values in 'constructionmaterial'
wood_materials = [
    'Timber Or Wood',
    'Mixed (Plaster And Timber)',
    'Mixed (Masonry And Timber)'
]
wood_houses = houses[houses['constructionmaterial'].isin(wood_materials)]
# Save to geojson
wood_houses.to_file('wood_houses_in_filtered_exeter_area.geojson', driver='GeoJSON')
summary = f"Filtered {len(wood_houses)} wood houses from {len(gdf)} buildings in the filtered Exeter area."
artifact_name = 'wood_houses_in_filtered_exeter_area.geojson'
artifact_description = (
    "GeoDataFrame of houses (single residential address) in the filtered Exeter area "
    "where the main construction material is wood or timber, saved as GeoJSON."
)
return [summary, artifact_name, artifact_description, wood_houses]
```

Filtered 864 wood houses from 69,169 buildings in the filtered Exeter area. The results are saved as 'wood_houses_in_filtered_exeter_area.geojson', which contains a GeoDataFrame of houses (single residential address) in the filtered Exeter area where the main construction material is wood or timber.

buildings_agent

Search Results: - 864 wood houses were found within the area defined by 'filtered_exeter_area_polygon'. - These are houses defined as buildings with buildinguse_addresscount_residential > 0 and buildinguse_addresscount_total = 1, where the main construction material is wood or timber.

Filtered Artifact: - wood_houses_in_filtered_exeter_area.geojson: Contains a GeoDataFrame of wood houses (single residential address) in the filtered Exeter area.

Let me know if you need further analysis or details from this dataset!

plotting_agent

Plot the wood houses in Exeter using the artifact 'wood_houses_in_filtered_exeter_area.geojson' and overlay the area boundary from 'filtered_exeter_area_polygon'. Provide a summary of the spatial distribution.

Tool Call: - Tool: generate_metadata_for_artifacts - Args: {"artifact_names": ["wood_houses_in_filtered_exeter_area.geojson", "filtered_exeter_area_polygon"]}

Tool call result: Metadata generated for 'wood_houses_in_filtered_exeter_area.geojson' and 'filtered_exeter_area_polygon'.

Tool Code Executor

```
import matplotlib
matplotlib.use('Agg')
def plot_wood_houses_exeter(data: list):
    import geopandas as gpd
    import folium
    import pandas as pd
    from shapely.geometry import mapping

    # Unpack data
    wood_houses = data[0].copy()
    exeter_boundary = data[1].copy()

    # Ensure CRS is EPSG:4326 for folium
    if wood_houses.crs is None or wood_houses.crs.to_epsg() != 4326:
        wood_houses = wood_houses.to_crs(epsg=4326)
    if exeter_boundary.crs is None or exeter_boundary.crs.to_epsg() != 4326:
        exeter_boundary = exeter_boundary.to_crs(epsg=4326)

    # Remove non-serializable columns (like Timestamps)
    def remove_non_serializable(df):
        for col in df.columns:
            if pd.api.types.is_datetime64_any_dtype(df[col]) or pd.api.types.is_timedelta64_dtype(df[col]):
                df[col] = df[col].astype(str)
        return df
    wood_houses = remove_non_serializable(wood_houses)
    exeter_boundary = remove_non_serializable(exeter_boundary)

    # Get centroid of Exeter for map centering
    centroid = exeter_boundary.geometry.centroid.iloc[0]
    m = folium.Map(location=[centroid.y, centroid.x], zoom_start=13, tiles='cartodbpositron')

    # Add Exeter boundary
    folium.GeoJson(
        exeter_boundary.geometry.iloc[0],
        name='Exeter Boundary',
        style_function=lambda x: {'color': 'blue', 'weight': 3, 'fillOpacity': 0.05}
    ).add_to(m)
```

```

# Add wood houses polygons
folium.GeoJson(
    wood_houses,
    name='Wood Houses',
    style_function=lambda x: {'color': 'green', 'weight': 1, 'fillOpacity': 0.4}
).add_to(m)

# Add layer control
folium.LayerControl().add_to(m)

# Save map
map_filename = 'wood_houses_exeter_map.html'
m.save(map_filename)

# Prepare summary
n_houses = len(wood_houses)
house_types = wood_houses['description'].value_counts().to_dict()
construction_materials = wood_houses['constructionmaterial'].value_counts().to_dict()
first5 = wood_houses[['osid', 'description', 'constructionmaterial', 'geometry_area_m2']].head(5)
summary = (
    f"Plotted {n_houses} wood houses in Exeter. "
    f"Most common house types: {house_types}. "
    f"Construction materials: {construction_materials}. "
    f"First 5 entries:\n{first5.to_string(index=False)}"
)

return [
    summary,
    'wood_houses_exeter_map',
    'A folium map showing the spatial distribution of wood houses in Exeter overlaid with the Exeter area boundary. Wood houses are shown in green polygons',
    map_filename
]

```

Plotted 864 wood houses in Exeter. Most common house types: {'Mid-Terrace House': 415, 'End-Of-Terrace House': 197, 'Detached House': 153, 'Semi-Detached House': 99}. Construction materials: {'Timber Or Wood': 799, 'Mixed (Plaster And Timber)': 34, 'Mixed (Masonry And Timber)': 31}. First 5 entries: osid description constructionmaterial geometry_area_m2 7c4c6a75-e59a-469c-8f34-396563bdf9ce Detached House Mixed (Plaster And Timber) 86.108 5e38cf8c-fd03-45bf-aeb6-a29ee0cd9e10 Mid-Terrace House Timber Or Wood 75.731 75aacd4c-6e0c-42b7-94ca-f1821d0ac514 Semi-Detached House Timber Or Wood 100.054 bff0291d-3b48-4790-a205-6481e62c8c35 Detached House Timber Or Wood 52.477 1bc8f299-eda6-46e8-aca8-73aa2d5c3b8a End-Of-Terrace House Timber Or Wood 69.094

Artifact generated: wood_houses_exeter_map

Description: A folium map showing the spatial distribution of wood houses in Exeter overlaid with the Exeter area boundary. Wood houses are shown in green polygons, and the city boundary in blue.

Map file: [wood_houses_exeter_map.html](#)

host_agent

There are 864 wood houses in Exeter, identified as houses with a main construction material of wood or timber. These include various types such as mid-terrace, end-of-terrace, detached, and semi-detached houses. The most common type is the mid-terrace house.

A map has been generated showing the spatial distribution of these wood houses within the Exeter area boundary. The wood houses are displayed as green polygons, and the city boundary is shown in blue.

If you need to view the map or want more details about specific locations or house types, let me know!